

SOFTWARE ARCHITECTURE · SEM 5

MedIntel

An AI-Powered Medical Assistant for Symptom Triage,
Health Records and Care Adherence

Layered Architecture · React.js · Node.js + Express · MongoDB · Gemini / OpenAI · REST

⚠ Clinical decision SUPPORT — not a diagnosis, not a doctor.

Presented by · Member 1 · Member 2 · Member 3 · Member 4

The Problem We Are Solving

Unreliable self-diagnosis

People search symptoms online and get contradictory, fear-inducing results with no personal context.

Pressure on primary care

Minor, self-limiting complaints occupy consultation slots, lengthening queues for those who truly need them.

Scattered health records

Prescriptions on paper, labs as PDFs in e-mail. History is lost exactly when a doctor needs it.

Poor medication adherence

Missed and mistimed doses silently undermine otherwise correct treatment.

These are four disconnected problems today — MedIntel joins them into one guided, record-keeping loop.

What MedIntel Is — and What It Is Not

✓ IT IS

- A guided symptom intake and triage assistant
- An explainer of likely conditions, in plain language
- A personal vault for reports, history and allergies
- A medication reminder and adherence tracker
- A router — it tells you how urgently to seek real care

✗ IT IS NOT

- A diagnosis — output is possibility, never verdict
- A prescription engine — no drug names, no dosages
- An emergency service — it escalates, it does not treat
- A replacement for a licensed clinician
- A store of record for legal or insurance purposes

⚠ THE GUARDRAIL PRINCIPLE

Every AI response is wrapped with a confidence band, a mandatory 'consult a physician' notice, and an urgency level. Red-flag symptom combinations bypass the AI entirely and trigger immediate escalation — a deterministic rule, not a probabilistic guess.

Functional Modules

01

User Authentication & Profile

Register / login, JWT sessions, hashed credentials, profile with age, sex, allergies, chronic conditions.

02

Symptom Analysis

Structured intake → red-flag rules → AI reasoning → ranked possible conditions with urgency level.

03

AI Health Chatbot

Follow-up conversation that remembers the session context and the user's stored history.

04

Medical Report Upload

PDF / image upload, OCR text extraction, AI summary of key values, attached to the timeline.

05

Medicine Reminder

Schedule doses, recurring jobs, push / e-mail notification, taken-or-missed adherence log.

06

Medical History

Unified chronological timeline of sessions, reports and medication, exportable for a real consultation.

Technology Stack — and Why

Layer	Technology	Why this choice
Presentation	React.js, Axios, Tailwind CSS	Component reuse across 6 modules; SPA keeps chat responsive without full reloads
API / Gateway	Node.js + Express.js	Non-blocking I/O suits an AI-call-heavy, I/O-bound workload; middleware chain for cross-cutting concerns
Business Logic	Express service modules	Plain, testable JS services — triage rules, orchestration, scheduling; no framework lock-in
AI Service	Gemini / OpenAI behind an adapter	Provider swapped by config, never by code change; enables fallback and cost comparison
Database	MongoDB (Atlas)	Symptom sessions and reports are nested, evolving documents — a poor fit for rigid relational rows
Cache / Jobs	Redis + node-cron	Caches repeated AI answers; drives reminder scheduling and OCR off the request path
File Storage	Cloud object storage (S3 / Cloudinary)	Binary reports do not belong in a database; CDN delivery and signed URLs
Security	JWT, bcrypt, Helmet, express-validator	Stateless auth for horizontal scaling; hashing, headers and validation at the edge
Deployment	Vercel (client) + Render / Cloud Run (API)	Managed CI/CD from Git, HTTPS by default, environment-scoped secrets

Architectural Style: Layered (N-Tier)

PRESENTATION LAYER

React SPA · Components, Router, State, Axios client, Form validation

API / GATEWAY LAYER

Express routes · REST endpoints, JWT verify, rate limit, schema validation, error envelope

BUSINESS LOGIC LAYER

Triage rules engine · Session orchestrator · Reminder scheduler · History service

AI SERVICE LAYER

Provider adapter · Prompt templates · PII redaction · Response validator · Circuit breaker

DATA ACCESS LAYER

Repositories · Mongoose models · Redis cache · Object storage client

CROSS-CUTTING

CONCERNS

- Authentication
- Authorization
- Input validation
- Logging & audit
- Error handling
- Configuration
- Caching
- Observability

Rule of the style: each layer calls only the layer directly below it — no skipping, no upward calls. → swap MongoDB for MySQL and only the Data Layer changes.

Layer Responsibilities & Contracts

Layer	Owns	Talks to	Must never
Presentation	Rendering, client-side validation, session token storage, UX state	API Layer over HTTPS/JSON only	Hold business rules or database credentials
API / Gateway	Routing, authN/authZ, request schema validation, rate limiting, error shaping	Business Logic Layer	Contain medical logic or query the database directly
Business Logic	Triage rules, orchestration, scheduling, history assembly, policy	AI Service + Data Access	Know about HTTP, req/res objects or the AI vendor
AI Service	Prompt building, redaction, provider calls, retries, response validation	External AI provider; returns DTOs	Persist data or expose vendor-specific types upward
Data Access	CRUD, queries, indexing, caching, file persistence	MongoDB, Redis, object storage	Apply business rules or return raw driver objects

Component View

CLIENT · React

- AuthPages (login / register)
- SymptomIntakeForm
- ChatWindow
- ReportUploader
- ReminderManager
- HistoryTimeline
- ApiClient (Axios + interceptors)
- AuthContext (token, guard routes)

SERVER · Express

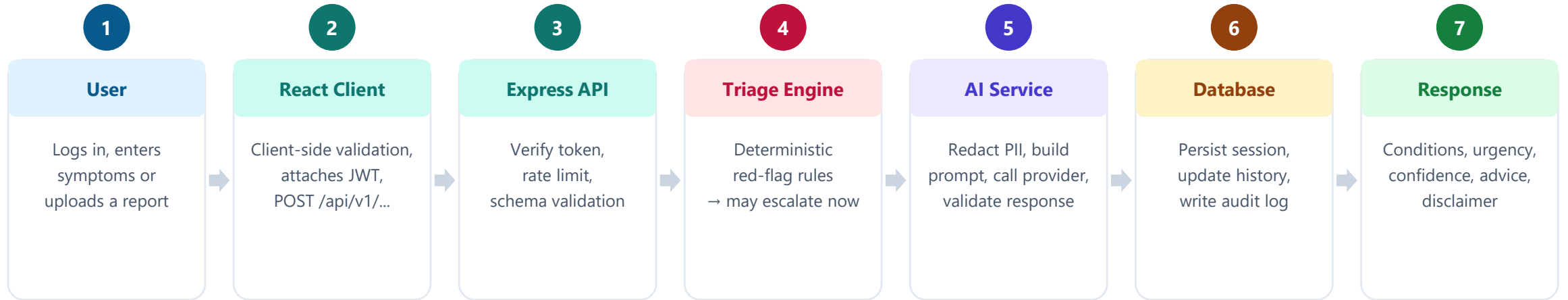
- authController + authService
- symptomController
- **TriageRulesEngine**
- AiOrchestrator
- reportService + OcrWorker
- reminderService + CronScheduler
- historyService
- middleware/ auth, validate, error, limit

DATA & EXTERNAL

- UserRepository
- SymptomSessionRepository
- ReportRepository
- ReminderRepository
- AuditLogRepository
- RedisCacheClient
- ObjectStorageClient
- AiProviderAdapter → Gemini / OpenAI

★ TriageRulesEngine is deterministic and runs before any AI call — the safety core of the system.

Runtime Workflow — Symptom Analysis



SHORT-CIRCUIT PATH

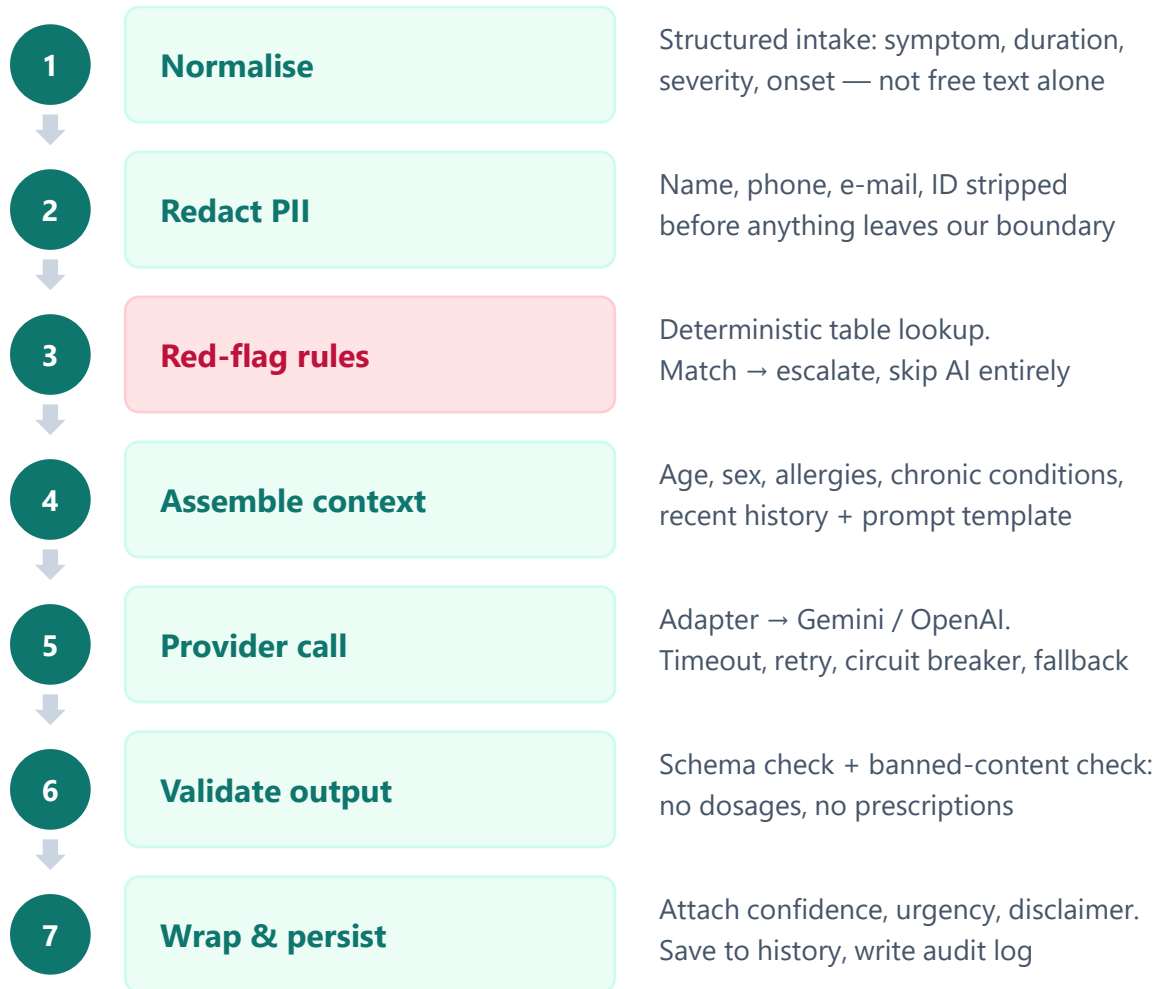
If step 4 matches a red flag (e.g. chest pain + breathlessness), the AI is never called — the user is told to seek emergency care immediately.

FAILURE PATH

If the AI provider times out, the circuit breaker opens: fall back to the second provider, then to rules-only guidance. The system degrades, it does not fail.

Long-running work — OCR of uploaded reports, reminder dispatch — is queued and processed asynchronously, never inside the request/response cycle.

Inside the AI Service Layer



WHY THIS SHAPE

Deterministic before probabilistic

Safety-critical decisions never depend on a model's mood.

Vendor-neutral by construction

The adapter is the only file that knows Gemini from OpenAI.

Fail-soft, never fail-open

Provider down → degraded rules-only advice, still safe.

Privacy at the boundary

Identifiers never reach a third-party API.

Auditable

Every AI input and output is logged for review.

Data Model

users

- `_id`, email (unique idx)
- passwordHash (bcrypt)
- profile: age, sex, allergies[]
- chronicConditions[]
- createdAt, lastLoginAt

symptom_sessions

- `_id`, userId (idx)
- input: symptoms[], duration, severity
- triageResult: urgency, redFlags[]
- aiResponse (immutable snapshot)
- confidence, createdAt

reports

- `_id`, userId (idx)
- fileUrl, mimeType, sizeBytes
- extractedText (OCR)
- aiSummary, keyValues{}
- uploadedAt

reminders

- `_id`, userId (idx)
- medicineName, dosageNote
- schedule: cron / times[]
- channel, active
- adherenceLog[] {at, taken}

chat_messages

- `_id`, userId, sessionId (idx)
- role: user | assistant
- content, tokensUsed
- createdAt (TTL optional)

audit_logs

- `_id`, userId, action
- resource, resourceId
- ipHash, userAgent
- aiProvider, latencyMs
- timestamp

Relationships by userId reference · indexed on userId + createdAt for timeline queries · audit_logs is append-only

REST API Design

Method	Endpoint	Purpose	Auth
POST	/api/v1/auth/register	Create account, hash password, issue JWT	Public
POST	/api/v1/auth/login	Authenticate, return access + refresh token	Public
POST	/api/v1/symptoms/analyze	Run triage → AI pipeline, persist session	JWT
GET	/api/v1/history?page=&limit=	Paginated unified timeline for the user	JWT
POST	/api/v1/reports	Multipart upload, queue OCR job	JWT
GET	/api/v1/reports/:id	Fetch report metadata, summary, signed file URL	JWT + owner
POST	/api/v1/chat/messages	Send follow-up message in a session context	JWT
GET / POST / PUT / DELETE	/api/v1/reminders/:id?	Full CRUD over medication reminders	JWT + owner

API CONVENTIONS

Versioned prefix /api/v1 · **JSON only** · **Bearer JWT** in Authorization header · **Uniform error envelope** { success, data, error: { code, message } } · **Correct status codes** (200 / 201 / 400 / 401 / 403 / 404 / 429 / 503) · **Pagination** on every list endpoint

Key Architectural Decisions

ID	Decision	Rationale	Trade-off accepted
ADR-01	Modular monolith, not microservices	One deployable unit fits a 4-person team and a semester timeline; module seams preserved	Whole app scales together; extraction work deferred
ADR-02	Deterministic triage engine in front of the AI	Safety-critical escalation must not depend on a probabilistic model	Rules table must be curated and maintained manually
ADR-03	MongoDB over MySQL	Sessions and reports are nested, schema-evolving documents; no heavy joins in our workload	Weaker cross-collection integrity; enforced in application code
ADR-04	AI provider behind an Adapter interface	Avoids vendor lock-in; enables fallback, A/B cost comparison and offline mocking in tests	One extra indirection layer to build and maintain
ADR-05	Stateless JWT authentication	No server session store → any instance serves any request; horizontal scaling is trivial	Token revocation needs a short TTL plus refresh-token rotation
ADR-06	Async queue for OCR and reminders	Keeps p95 request latency low; slow work never blocks the user's thread	Adds a worker process and eventual-consistency semantics

Quality Attributes & Architectural Tactics

Attribute	Measurable scenario	Architectural tactic
Safety	100% of defined red-flag combinations escalate, even if the AI is unavailable	Deterministic rules engine evaluated before the AI call; rules-only fallback path
Privacy	No direct identifier ever leaves our trust boundary	PII redaction stage in the AI layer; encryption at rest; append-only audit log
Performance	p95 response for symptom analysis under 3 s at 100 concurrent users	Redis cache for repeated queries; async offload of OCR; indexed queries; streaming responses
Availability	Core triage stays usable when a single AI provider fails	Circuit breaker, provider fallback, graceful degradation to rules-only mode
Modifiability	Swap AI vendor or database with changes confined to one layer	Adapter and Repository patterns; strict layer dependency rule; DTOs at every boundary
Scalability	Handle traffic growth without re-architecting	Stateless API instances behind a load balancer; managed DB with connection pooling
Security	Withstand credential stuffing and injection attempts	bcrypt hashing, rate limiting, schema validation at the edge, Helmet headers, RBAC
Usability	A non-technical user completes an analysis in under 90 seconds	Guided structured intake instead of free-text; plain-language output with urgency colour coding

Design Patterns Applied

Layered

Overall system decomposition into five stacked layers

Adapter

AiProviderAdapter — one interface, Gemini or OpenAI behind it

Repository

UserRepository, ReportRepository — hides Mongoose from services

Strategy

Interchangeable prompt strategies per symptom category

Chain of Responsibility

Express middleware pipeline: auth → validate → limit → handle

Facade

AiOrchestrator exposes one call over a seven-stage pipeline

Observer / Pub-Sub

Reminder events fan out to notification channels

Circuit Breaker

Trips after repeated AI provider failures; opens the fallback path

DTO

Typed objects crossing every layer boundary — no leaking internals

Each pattern is justified by a quality attribute — Adapter serves modifiability, Circuit Breaker serves availability, Repository serves testability.

Security & Privacy Architecture

IDENTITY

- bcrypt password hashing with salt rounds
- JWT access token, short TTL
- Refresh-token rotation on use
- Role-based access: user / admin
- Ownership check on every resource

TRANSPORT & EDGE

- HTTPS / TLS enforced end to end
- Helmet security headers
- Strict CORS allow-list
- Rate limiting per IP and per user
- Schema validation before any handler

DATA

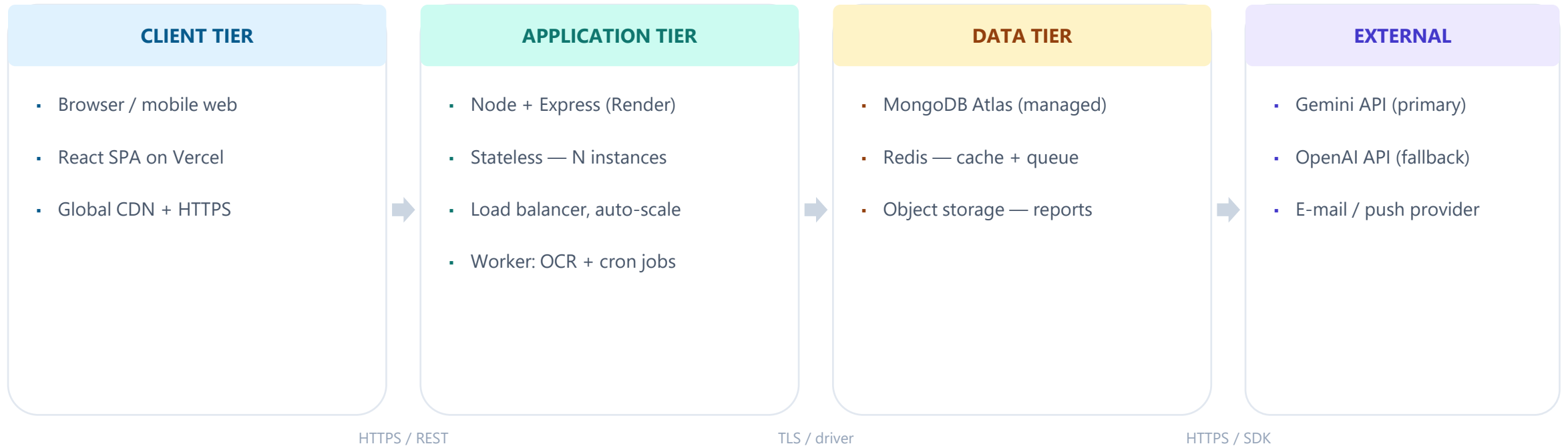
- Encryption at rest on the managed DB
- Signed, expiring URLs for report files
- Secrets in environment variables only
- Append-only audit log of all access
- Right to delete: full account purge

AI BOUNDARY

- PII redacted before every provider call
- No raw report images sent to the AI
- Prompt-injection filtering on user text
- Output validated against a schema
- Provider traffic logged for audit

⚠ Design principle: the AI provider is treated as an untrusted external party. Identifiable data never crosses that boundary.

Deployment View



DELIVERY PIPELINE

Git push → **automated build & tests** → preview deployment per branch → **review** → promote to production. Environment-scoped secrets; no credentials in the repository; rollback is a one-click redeploy of the previous build.

Risks & Mitigations

Risk	Impact	Mitigation
AI produces an incorrect or hallucinated suggestion	High · Safety	Rules engine owns escalation; output schema + banned-content validation; confidence shown; mandatory disclaimer
User treats output as a diagnosis and skips real care	High · Safety	Urgency-first UI, persistent disclaimer, explicit 'see a doctor' call-to-action on every result
AI provider outage, rate limit or cost spike	Medium · Availability	Circuit breaker, secondary provider fallback, Redis response cache, per-user quotas
Breach or leakage of sensitive health data	High · Privacy	PII redaction at the boundary, encryption at rest and in transit, RBAC, audit logging, least privilege
Latency exceeds user patience on AI calls	Medium · Performance	Streamed partial responses, cached common queries, async offload of OCR, p95 budget tracked
Prompt injection through free-text symptom input	Medium · Security	Structured intake over free text, input sanitisation, system-prompt isolation, output validation
Scope creep across six modules in one semester	Medium · Delivery	Phased roadmap; modules 1–3 form the graded core, 4–6 follow incrementally

Roadmap & Future Enhancements

PHASE 1

This semester — committed

- Authentication & profile
- Symptom analysis + triage engine
- AI chatbot with session context
- Medical history timeline
- Deployed, documented, tested

PHASE 2

Next iteration

- Report upload + OCR extraction
- Medicine reminders & adherence
- Multilingual support (regional languages)
- Doctor appointment booking
- Nearby hospital & pharmacy lookup

PHASE 3

Long term vision

- Emergency SOS with location sharing
- Wearable device integration (vitals)
- Extract AI layer into its own service
- Event-driven core for real-time alerts
- FHIR-compliant record exchange

None of these require re-architecting — the module seams (ADR-01) and the provider adapter (ADR-04) were chosen precisely to absorb this growth.

Team Responsibilities

MEMBER 1

Project Overview & Requirements

Problem framing, scope and safety boundary, module definition, requirement documentation.

Slides 1–4

MEMBER 2

Architecture & Technology

Layered design, layer contracts, component view, stack justification, deployment view.

Slides 5–8, 17

MEMBER 3

Workflow, AI Pipeline & Data

Runtime flow, AI safety pipeline, data model, REST API contract.

Slides 9–12

MEMBER 4

Decisions, Quality & Risk

ADR log, quality attributes and tactics, design patterns, security, risks, roadmap.

Slides 13–16, 18–19

MedIntel makes healthcare guidance faster and better organised — while keeping the clinician, not the algorithm, in charge of the diagnosis.

Thank You

Questions and feedback welcome.

MedIntel · Software Architecture, Semester 5